SereneDB challenge · SchubKI Hackathon 2026

Making Sense of Logs

Ask your logs a question. Get the answer your role needs, fast, cheap, and with proof.

Amirali & Konark

14 s

per answer

≈ 5 ¢

per answer

$0.02 · 20 s

to judge 1,126 kinds of message

THE PROBLEM

Millions of lines, three people, no answers



Engineer

“What failed?”



Team lead

“What keeps coming back?”



Manager

“What do we fix first?”

4.7 M

lines: no prompt fits

855,501

FATAL lines, 36,714 bursts

86 s

per answer on raw files

THE PIPELINE

The database counts, the AI explains

Load

SereneDB parses and indexes

72 s · 5 M lines

Judge

Jev labels each kind of message

$0.02 · 20 s

Ask

Claude picks MCP tools

< 0.1 s per query

Answer

written for your role

14 s · ≈ 5 ¢

Verify

every number re-runs

62 / 62 traced

Setup: docker compose up -d · uv sync · claude · “Load the base corpus”

STEP 1 · LOAD

SereneDB loads and indexes every line

1 · Load | 2 · Judge | 3 · Ask | 4 · Answer | 5 · Verify

- Download from Zenodo, or any log file on disk
- Format detected from 76 known formats
- Parsed with SQL inside SereneDB: typed columns
- Masked into kinds of message; bursts by window functions
- One inverted index: full text, BM25, phrase, fuzzy

72 s
clean clone → BGL + OpenStack ready (5 M lines)

18 s
4.7 M BGL lines parsed + indexed in SQL

1.6 s
a new 21 k-line file, format auto-detected

STEP 2 · JUDGE

Jev decides what is a real problem, once

1 · Load | **2 · Judge** | 3 · Ask | 4 · Answer | 5 · Verify

- Typed questions, calibrated probabilities: real fault? routine dump? attack? whose?
- Asked once per kind of message, inside SereneDB with one SQL call
- Stored: answering never re-reads raw lines
- Real alerts in the manager's top 10: 1 → 5

Judge BGL once	cost	time
Jev, per kind of message	**$0.02**	**20 s**
Haiku 5.5 reads all lines	**$220**	**≈ 2.5 h**
Sonnet 5.5 reads all lines	**$880**	**≈ 2.5 h**
Opus 5.5 reads all lines	**$1,760**	**≈ 2.5 h**

LLM: BGL ≈ 440 M tokens (measured 1.7 B/token), ≥ 440 calls of 1 M; ≈ 48 k tokens/s measured. Again for every question.

STEP 3 · ASK

Claude asks focused questions through MCP

1 · Load | 2 · Judge | **3 · Ask** | 4 · Answer | 5 · Verify

- 44 tools: search, context, counts, trends, rankings, owners
- Each tool is a fixed, safe SQL query; Claude fills typed fields
- Results: the SQL, exact totals, ≤ 50 rows, truncation flag
- 5 prompts (type /) and 6 resources (type @)

7 ms
full-text count over 4.7 M lines (inverted index)

3–39 ms
group-bys straight from the term dictionary

1.8 s
of database time for a 20-step investigation

STEP 4 · ANSWER

One problem, three different answers

1 · Load | 2 · Judge | 3 · Ask | **4 · Answer** | 5 · Verify

- Engineer: 12,996 lines on 1,024 nodes, one host → check it
- Team lead: 31 bursts, owner disputed → triage together
- Manager: rank 1 of 133 → name an owner now
- Facts, hypothesis, unknowns and a next step, every time

14 s

per answer with Claude Code + our server

≈ 5 ¢

per answer (Claude Code, 50 questions)

STEP 5 · VERIFY

Every number traces to its query

1 · Load | 2 · Judge | 3 · Ask | 4 · Answer | **5 · Verify**

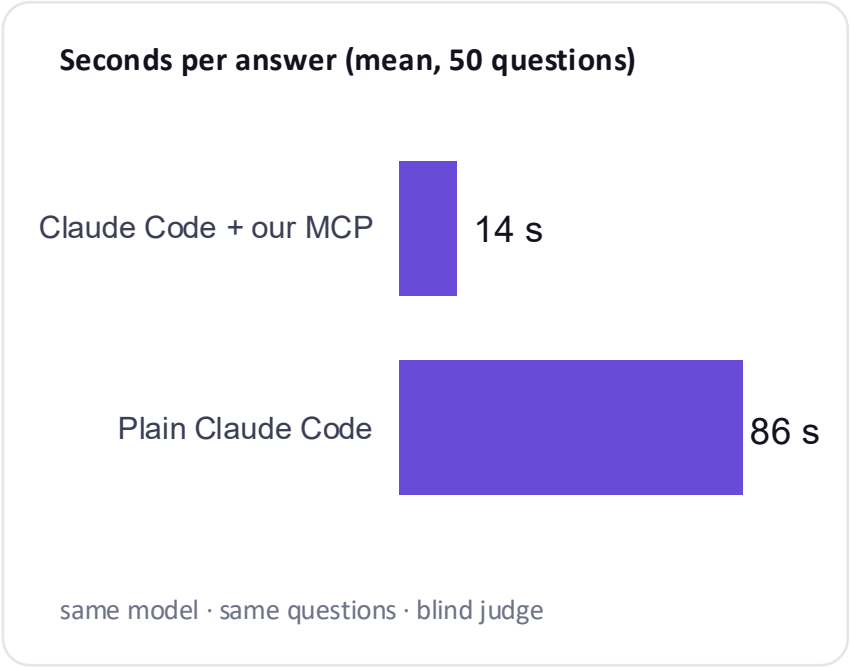
- Session report: each number links to the step that produced it
- Verify re-runs the step read-only and compares
- Jev checks each sentence against its evidence
- Offline copy: docs/investigation/report.html

62 / 62
numbers in the worked investigation traced to a query

17 / 17
steps give the same result when re-run

PRICE & PERFORMANCE

Faster, more correct, for about a cent more



6× faster — 14 s vs 86 s per answer

4.9 ¢ vs 3.8 ¢ — per answer: about a cent more

47 vs 100 — wrong claims per dollar spent

< 1 s — of each answer is the database; the rest is AI

RESULTS

Measured on 50 questions with known answers

30 – 15

wins vs plain Claude Code (18 – 4 on BGL + OpenStack)

116 vs 186

wrong claims

64 vs 46 %

key facts found

Where it still loses: engineer questions on small logs (8 – 9), where grep can read every line. All 50 answers and the judge's reasons are in the dashboard.

FEATURES

Each feature fixes a way answers go wrong

Bursts, not lines

a storm of 12,996 lines reads as 31 recurrences

Jev judgments

real alerts in the top 10: 1 → 5

Exact totals in SQL

62 / 62 numbers traced; no rounding by the model

Ownership table

every problem has a team; 4 disputes flagged

Computed priority

the manager's ranking is SQL, not opinion

Facts vs hypotheses

unknowns stated in 100 % of answers

BEYOND THE BRIEF

More than the challenge asked for

Your own logs

any file, no code

1 B remote rows

queried in place on S3

Access per team

enforced by the database

11.2 M lines

HDFS loaded in 47 s

Blind evaluation

50 answer keys + dashboard

Fully reproducible

--fresh reruns Jev live, same result

SCALE

Where it goes next



Mittelstand

production and IT logs



Other industries

anything that writes logs



A product

a self-hosted log copilot

10

log types, no code change

≈ 5 ¢

per answer

2 min

fresh install → answers

Try it in two minutes

```
docker compose up -d
uv sync
claude
> Load the base corpus
```

github.com/aircode610/making-sense-of-logs-schubki-ai-hackathon